

Software Evolution

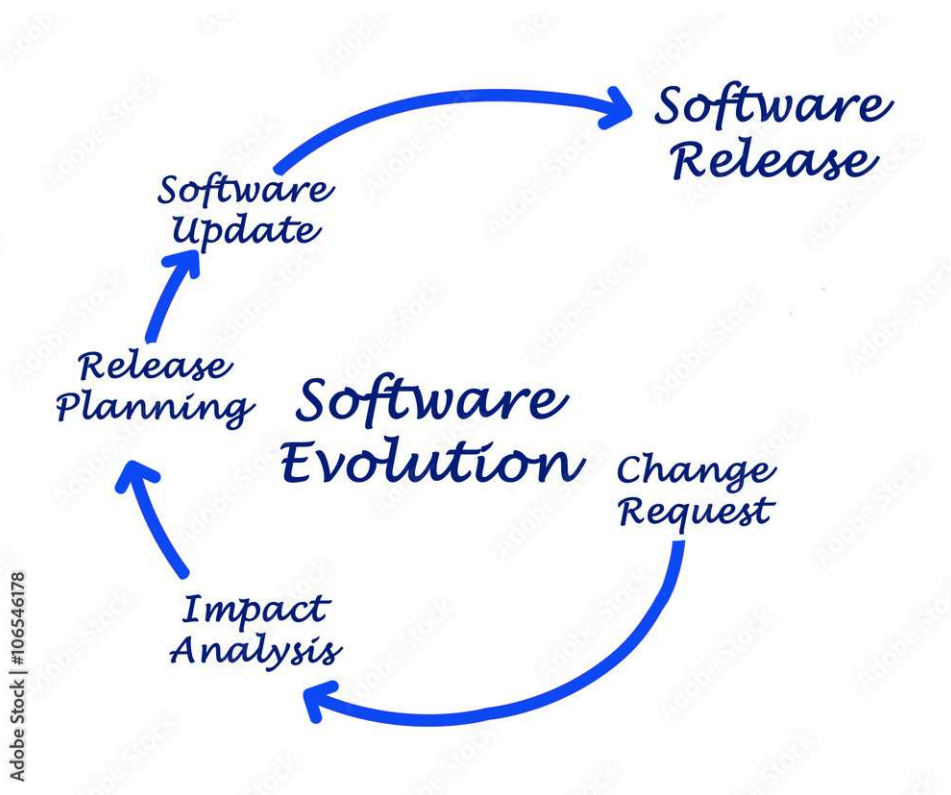
Software evolution is the process of **maintaining, updating, and improving a software system after it has been deployed**. Software is rarely “finished” at release; it evolves over its lifetime to meet changing requirements, fix errors, adapt to new environments, or improve functionality.

Reasons for Software Evolution:

- **Requirement changes:** Users may want new features.
- **Environment changes:** Updates in hardware, operating systems, or platforms.
- **Errors or security issues:** Bugs or vulnerabilities need fixing.
- **System improvements:** Performance or usability enhancements.

Software Evolution Process:

1. **Change Request:** Users or stakeholders request changes to the system.
 - Example: Adding an online payment feature to an existing shopping app.
2. **Impact Analysis:** Evaluate how the changes affect the system, including cost, effort, and risk.
 - Example: Checking if adding payments affects inventory, order processing, or security.
3. **Release Planning:** Decide which changes will go into the next system version.
4. **System Update:** Implement the changes and test them thoroughly.
5. **System Release:** Deploy the updated version to users.



Software evolution ensures that a system **remains useful, secure, and efficient over time**, responding to both user needs and technological changes.

Example in real life: Updating WhatsApp to add new features like “Status” or encryption is part of its software evolution.

Legacy Systems

A **legacy system** is a computer system, software application, or technology that is **outdated**, may no longer receive official support or maintenance, but is **still essential for an organization**. These systems often cannot be easily replaced due to business dependence or high replacement risk.

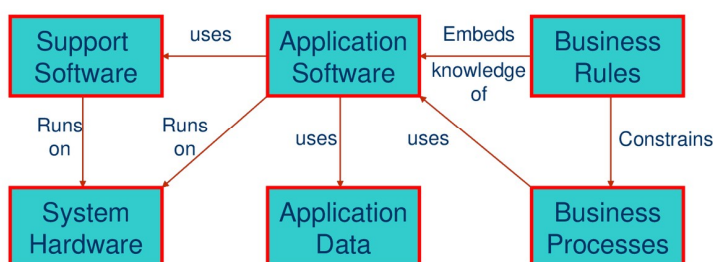
Banking Core Systems:

- Banks use **COBOL programs running on mainframes** to process deposits, withdrawals, transfers, loan calculations, and other financial operations.
- Mainframes can support thousands of users simultaneously (ATMs, online banking, branch systems).
- Even modern banking apps often **connect to the legacy COBOL mainframe in the background** to process transactions.
- Example: The **New York Stock Exchange** and many large banks use legacy systems to handle transactions because they are stable and critical for business, even though the technology is outdated.

Components of Legacy Systems:

- **Application Software:** Main programs that perform business functions.
Example: In a bank, the core banking software handles account management and transactions.
- **Support Software:** Operating systems or middleware that run the applications.
Example: The mainframe OS that runs the bank's transaction processing software.
- **System Hardware:** Physical machines on which the software runs.
Example: Mainframe computers used by the bank to process customer transactions.
- **Application Data:** Data used and produced by the system.
Example: Customer account balances, transaction histories, and loan records.
- **Business Processes & Rules:** Knowledge and procedures embedded in the system that support operations.
Example: Bank rules for calculating interest or approving loans automatically.

Legacy system components



Docsity.com

Strategies for Evolving Legacy Systems:

1. **Scrap the system completely:** Used when the system no longer effectively supports business processes.
 - Example: Retiring an old inventory system that is no longer accurate.
2. **Reengineer the system:** Improve maintainability and functionality without a full replacement.
 - Example: Updating code or database structures in a payroll system while keeping the core software.
3. **Leave the system unchanged with regular maintenance:** Chosen when the system is stable

and only minor updates are needed.

- Example: Maintaining an old reporting system that rarely needs changes.

4. Replace part of the system: Necessary when hardware is obsolete or off-the-shelf solutions are cost-effective.

- Example: Moving from a custom-built accounting system to a modern ERP system.

This approach allows organizations to **balance risk, cost, and functionality** when dealing with essential but outdated systems.

Software Maintenance

Software maintenance is the activity of modifying a software system after it has been delivered, in order to **correct errors, improve performance, remove obsolete features, or enhance existing features**. It usually does **not involve major changes to the system's architecture**, but focuses on modifying or adding components.

Software evolution is about continuously improving and adapting software over its lifetime, while software maintenance is about keeping existing software functional and fixing or optimizing it.

Maintenance Activities:

- 1. Obtaining Maintenance Requests:** Users submit change requests through a formal process.
 - Example: A customer reports that an online shopping app crashes when applying a discount code.
- 2. Transforming Requests into Changes:** Analyze the request to determine its scope, impact, and required time.
 - Example: Checking if the discount module affects the checkout, inventory, or payment systems.
- 3. Designing Changes:** Convert the request into a formal design change.
 - Example: Planning code modifications to fix the discount bug and ensure correct calculations.
- 4. Implementing Changes:** Apply the approved design changes to the system components.
 - Example: Updating the discount calculation logic and testing it.

Types of Maintenance:

- 1. Corrective Maintenance:** Fixes errors or defects in the system.
 - Example: Repairing a login module that sometimes rejects valid credentials.
- 2. Adaptive Maintenance:** Updates parts of the system affected by changes elsewhere.
 - Example: Modifying the reporting module after a database schema change.
- 3. Perfective Maintenance:** Enhances functionality or performance.
 - Example: Adding a faster search feature in an e-commerce website.
- 4. Preventive Maintenance:** Takes actions to prevent future errors.
 - Example: Refactoring code to reduce the chance of future bugs.

Cost of Maintenance:

Maintenance can be expensive, often **60–80% of the software budget** in organizations.

Factors Affecting Maintenance Cost:

- **Number of defects:** More unknown errors increase cost.
- **Number of users/customers:** More users mean more issues and requests.
- **Quality of documentation:** Good documentation reduces maintenance effort.
- **Tools:** Tools for automated documentation or testing lower costs.
- **Well-structured programs:** Easier-to-understand and well-designed programs reduce maintenance time.

Example in Real Life:

A banking system might need corrective maintenance to fix transaction errors, adaptive maintenance when updating to a new operating system, perfective maintenance to speed up

customer account searches, and preventive maintenance to improve code quality for future changes.